

8. Write a program to simulate Banker's Algorithm Safety Algorithm.

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int main() {
```

```
    int n, m;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int allocation[n][m], max[n][m], need[n][m];
```

```
    int available[m];
```

```
    printf("Enter allocation matrix: \n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            scanf("%d", &allocation[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Enter maximum Demand Matrix: \n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            scanf("%d", &max[i][j]);
```

```
        }
```

```
    }
```

```

printf("Enter available resources: \n");
for (int i=0; i<m; i++) {
    scanf("%d", &max[i][j]); available[i]};
}
}

```

```

for (int i=0; i<n; i++) {
    for (int j=0; j<m; j++) {
        need[i][j] = max[i][j] - max allocation[i][j];
    }
}

```

```

int work[m];
bool finish[n];
int safeSeq[n];

```

```

for (int i=0; i<m; i++)
    work[i] = available[i];

```

```

for (int i=0; i<n; i++)
    finish[i] = false;

```

```

int count = 0;

```

```

while (count < n) {
    bool found = false;

```

```

    for (int i=0; i<n; i++) {
        if (!finish[i]) {
            bool canExecute = true;

```

```

            for (int j=0; j<m; j++) {

```

if (need[i][j] > work[j])
canExecute = false;
break;

}

}

if (canExecute) {

for (int j = 0; j < m; j++)

work[j] += allocation[i][j];

safeSeq[count++] = i;

finish[i] = true;

found = true;

}

}

}

if (!found) {

printf("In System is in an UNSAFE state (Deadlock possible).\n");

return 0;

}

}

printf("In System is in a safe state.\n");

printf("Safe Sequence is:");

for (int i = 0; i < n; i++) {

printf("%d", safeSeq[i]);

if (i != n - 1)

printf(" -> ");

}

printf("\n");

return 0;

}

output:

Enter number of processes: 5

Enter number of resources: 4

Enter allocation matrix:

1	2	3
3	4	2
5	2	6
2	0	1
0	0	1

Enter maximum demand matrix:

4	5	3
3	4	2
3	5	6
2	1	1
1	2	3

Enter Available resources:

5	5	5
---	---	---

System is in a safe state.

Safe Sequence is $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$

Enter allocation matrix:

0	0	1	2
1	0	0	0
1	3	5	4
0	6	3	2
0	0	1	4

Enter maximum demand matrix:

0 0 1 2
1 2 5 0
2 3 5 6
0 6 5 2
0 6 5 6

Enter available resources:

1 5 2 0

System is in a safe state.

Safe sequence is: $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

	Allocation				Demand				Available				Need			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	2	1	5	2	0	0	0	0	0
P_1	1	0	0	0	1	7	5	0					0	7	5	0
P_2	1	3	5	4	1	3	5	6					1	0	0	2
P_3	0	6	3	2	0	6	5	2					0	0	2	0
P_4	0	0	1	1	0	6	5	6					0	6	4	2
	1	5	2	0												
	3	14	10	12												

P_0 finish[0] = false

~~need~~ $\leq$ ^{available} allocation

available = max-allocation

0000 $\leq$ 1520

work = allocated + work

= 1520 + 0012

= 1532

finish[0] = true.

P_1 finish[1] = false

need $\leq$ available.

28/04/26

$\{P_0,$